OPERATOR STUDIO

Mentoria *Operator*

05

SEMANA 05

Vendas, CRM e captação automática

Você sai com o funil desenhado e um CRM próprio, numa página privada do seu site, com os clientes atuais importados e cada novo contato entrando sozinho, com aviso para você e follow-up automático.

ENTREGA	Funil com etapas e critérios · CRM privado em /crm/ no seu site, protegido por senha, com os clientes atuais · Leads do formulário e do agendamento entrando sozinhos, com consentimento registrado · Aviso de novo lead pelo Make · Follow-up automático testado
FERRAMENTAS	Claude · GitHub · Netlify (Functions, Blobs, funções agendadas, variáveis de ambiente) · Make.com · Gmail · WhatsApp Business · API do Claude (opcional)
TEMPO	3 a 5 horas

MATERIAL DO ALUNO · USO INDIVIDUAL

Antes da sessão

Contas e acessos estão no **Guia da Semana 0**. Confira e traga:

- **O site da Semana 3 no ar**, com o repositório no GitHub ligado ao Netlify e a função do formulário (ou do agendamento) funcionando.
- **Make.com** com a conexão do Gmail do negócio criada. Confira o fuso horário da organização no Make.
- **A lista de clientes atuais exportada em CSV**, de onde ela estiver (sistema antigo, contatos do celular). Se estiver no papel, digite nome, WhatsApp, e-mail, o que comprou e quando; o Claude transforma em CSV.
- **Os últimos 10 contatos que chegaram** e o que aconteceu com cada um: fechou, sumiu, pediu orçamento, ficou de pensar.
- **Seu Projeto no Claude** (Semana 1) e o **Brand Kit** (Semana 2).
- **Três decisões:** prazo para responder um lead novo, quantos lembretes no máximo e a senha do CRM (digitada só no Netlify e no login do CRM, nunca no chat).
- **Opcional (perfil Automatizar):** conta no Console da Anthropic (platform.claude.com) para o relatório automático de reunião.

O conceito em 10 minutos

Venda perdida quase nunca é falta de cliente. É contato que ninguém respondeu a tempo, orçamento que ninguém cobrou, cliente antigo que ninguém chamou de volta.

O CRM resolve isso com uma regra: **todo contato vira um registro, todo registro tem uma etapa, toda etapa tem uma próxima ação com data.** Se você abre o CRM e sabe quem precisa de você hoje, ele está funcionando.

Nesta mentoria o CRM é uma **página privada do seu próprio site**, construída pelo Claude no mesmo repositório do GitHub e publicada pelo Netlify. Os dados e o desenho do funil são seus, e mudar é só pedir ao Claude.

É assim que o CRM da Operator Studio funciona: página /crm/ no mesmo site, com senha. Cada reserva do site entra sozinha como lead, e dá para adicionar leads à mão. Cada lead tem etapa, notas, valor, arquivos anexados (propostas, contratos), follow-up e, depois da reunião, um relatório gerado pela API do Claude a partir da transcrição.

As peças, em linguagem simples:

```
Cliente preenche o formulário do site
   → Netlify Function (Semana 3) confere os dados e o consentimento
   → grava o lead no Netlify Blobs (o "banco de dados" do site)
   → chama o webhook do Make → Gmail avisa você

Você abre seudominio.com.br/crm/ e digita a senha
   → a função /api/crm confere a senha (CRM_KEY) e devolve os leads
```

```
Todo dia de manhã, uma função agendada procura quem precisa de follow-up
   → chama o Make → Gmail envia a mensagem ao lead
```

Três cuidados separam um sistema confiável de um que você abandona em um mês: **consentimento** (LGPD), **gravar primeiro e avisar depois** (se o aviso falhar, o lead já está salvo) e **teste como cliente**, do celular, do formulário ao follow-up.

Passo a passo

1. Desenhe as etapas do funil

Antes de construir, feche o funil. Sistema sem processo só organiza a bagunça.

1. No seu Projeto do Claude, use o **Prompt 5.1** com os seus últimos 10 contatos.
2. O Claude devolve de 5 a 7 etapas com critério de entrada, próxima ação e prazo. Exemplo de um salão:

ETAPA	CRITÉRIO DE ENTRADA	PRÓXIMA AÇÃO	PRAZO
Novo lead	Chegou pelo site, WhatsApp ou Instagram	Responder	2 horas úteis
Em conversa	O cliente respondeu	Entender o que precisa	2 dias
Proposta enviada	Valores enviados	Cobrar retorno	3 dias
Agendado	Data marcada	Confirmar na véspera	Até o atendimento
Cliente	Serviço feito e pago	Pedir avaliação e retorno	7 dias
Perdido	Recusou ou não respondeu ao último follow-up	Registrar motivo	No mesmo dia

3. Teste: **consigo dizer em qual etapa está cada um dos 10 contatos?** Se algum não cabe, falta etapa ou um critério está vago.
4. Defina os **motivos de perda** (preço, prazo, sem resposta, escolheu outro, fora do perfil). Eles mostram onde você perde venda.

Para comparar: o funil da Operator Studio é Novo, Análise agendada, Análise feita, Proposta, Cliente e Perdido.

2. Defina o modelo de dados

O **Netlify Blobs** guarda os dados do site. Pense num arquivo de gavetas: cada gaveta (o Netlify chama de *store*) guarda fichas, e cada ficha tem um código e um conteúdo. Cada lead é uma ficha na gaveta `leads`; os anexos ficam na gaveta `crm-arquivos`.

1. Cole o funil aprovado no **Prompt 5.2**. O Claude devolve a ficha do lead com cada campo, o tipo e para que serve.
2. A ficha mínima tem: contato padronizado (nome, WhatsApp, e-mail), origem (site, agendamento, manual, importado), serviço e mensagem, etapa, valor, notas, próxima ação com data, último contato, número de follow-ups enviados, **Não contatar**, motivo de perda, **consentimento** (sim ou não, data e hora, texto exato do checkbox, página), anexos e datas de criação e atualização.
3. **Minimização de dados (LGPD):** nada de CPF, data de nascimento, endereço ou dado de saúde, a menos que o seu serviço precise de verdade.

4. Salve a ficha aprovada nos conhecimentos do Projeto. Todos os prompts seguintes usam esses nomes exatos.

No CRM da Operator Studio, as reservas do site ficam numa gaveta e os dados do funil (etapa, notas, valor) noutra, ligados pelo mesmo código do lead. Para começar, uma gaveta `leads` é mais simples.

3. Crie a senha e a API do CRM

O CRM tem duas partes: a **API** (uma Netlify Function em `/api/crm`, que lê e grava no Blobs) e a **página** (`/crm/`). A senha protege a API: sem ela, nada sai.

Guardar a senha no Netlify (variável de ambiente):

1. No painel do Netlify, abra o projeto e vá em **Project configuration** → **Environment variables** (em algumas contas, Site configuration).
2. Clique em **Add a variable** e escolha adicionar uma variável (a outra opção importa de arquivo). **Key** = `CRM_KEY`, **Value** = a senha: uma frase longa, de pelo menos 16 caracteres, que você não usa em outro lugar.
3. Salve. Variável nova só vale depois de um novo deploy: em **Deploys**, publique de novo, ou espere a próxima atualização pelo GitHub.

Construir a API:

1. Cole o **Prompt 5.3**. O Claude cria `netlify/functions/crm.mjs`, acrescenta o pacote `@netlify/blobs` ao `package.json` e atualiza o GitHub. O Netlify publica sozinho.
2. Teste de segurança: chamada à `/api/crm` sem senha e com senha errada. As duas respondem **401 (não autorizado)**.

A senha viaja num cabeçalho da requisição, nunca no endereço. Ela não está no código nem no GitHub: a função lê `CRM_KEY` do Netlify na hora.

4. Ligue o formulário e o agendamento ao CRM, com consentimento

A função do formulário da Semana 3 passa a gravar o lead no Blobs, com consentimento, e a avisar você.

Consentimento (LGPD):

1. Use o **Prompt 5.5** para o texto do checkbox e a página `/privacidade`.
2. O checkbox é **obrigatório**, **vem desmarcado** e tem link para a política. Exemplo: "Autorizo [NOME DO NEGÓCIO] a falar comigo por WhatsApp e e-mail sobre este pedido. Li a Política de Privacidade."
3. A função confere de novo no servidor e guarda, junto com o lead, o texto exato aceito, a data e hora e a página. Se o texto mudar no futuro, cada registro mantém o que a pessoa aceitou.

A função:

1. Cole o **Prompt 5.4**. O Claude lê a função atual no repositório e acrescenta o CRM.

2. A ordem dentro da função: validar → gravar no Blobs → responder ao cliente → avisar pelo Make. Gravar vem antes de avisar.
3. Mesmo e-mail ou WhatsApp de novo? A função **atualiza** o lead existente, guarda a nova mensagem nas notas e manda o aviso "lead voltou".
4. A armadilha contra robôs da Semana 3 continua: se o campo escondido vier preenchido, a função responde "ok" e não grava nada.
5. Agendamento também vira lead, com a data marcada. No site da Operator Studio, cada reserva da análise gratuita entra no CRM sozinha, enquanto o Make cria o evento no Google Agenda com Meet e manda a confirmação.

Depois do deploy, envie um teste. No Netlify, em **Data & Storage** → **Blobs**, abra a gaveta `leads`: a ficha do teste precisa estar lá. Se não estiver, abra os logs da função e cole o erro no Claude.

Os dados do Blobs valem para o site inteiro, não para cada versão publicada: teste também vai para o CRM de verdade. Use "TESTE" no nome e apague no fim.

5. Construa a página /crm/

1. Cole o **Prompt 5.6**. O Claude cria `crm/index.html` com o Brand Kit e a função de anexos.
2. O que a página tem: - **Login** com senha, lembrada só neste aparelho, e botão **Sair**. - **Hoje:** próxima ação para hoje ou atrasada. A tela de toda manhã. - **Funil:** uma coluna por etapa, com contagem; no celular, abas. - **Ficha do lead:** contato, origem, consentimento, etapa, notas, valor, próxima ação com data, motivo de perda e **Não contatar**. - **WhatsApp com um toque**, com mensagem pronta (link `wa.me`), como as mensagens rápidas do CRM da Operator Studio. - **Novo lead manual** e **Busca**. - **Anexos:** propostas e contratos na gaveta `crm-arquivos`, com limite de tamanho (no CRM da Operator Studio, PDF ou imagem de até 5 MB).
3. A página não aparece no Google: o Claude coloca a instrução de não indexar na página e no `netlify.toml`.
4. No celular, abra `seudominio.com.br/crm/`. Senha errada é recusada. Com a certa, mova o lead de teste, escreva uma nota, anexe um PDF, saia e entre de novo: tudo continua lá.

6. Importe os clientes atuais pelo CSV

1. Use o **Prompt 5.7** com o CSV, só com as colunas que vão para o CRM. O Claude padroniza telefones, e-mails e nomes, remove duplicados e devolve um CSV limpo.
2. Confira 10 linhas aleatórias contra a lista original. O Claude pode errar um DDD ou juntar duas pessoas com o mesmo nome. **Revisão humana obrigatória.**
3. No mesmo prompt, o Claude cria no CRM o botão **Importar CSV**: você escolhe o arquivo, vê a prévia (novos, já existentes, com problema) e só então confirma.
4. Clientes antigos entram com etapa **Cliente** e origem **importado**. Consentimento só para quem de fato autorizou, com nota de como. Na dúvida, desmarcado.
5. Depois, apague o CSV do computador e a conversa com a lista.

7. Monte o aviso de novo lead no Make

O padrão da Operator Studio: o site monta o e-mail e manda para um webhook do Make, e o Make envia pelo Gmail. Um cenário serve para todos os avisos.

```
[1 Webhooks: Custom webhook]
        |
[2 Router]
   |-- tipo = novo_lead, lead_voltou ou erro --> [3 Gmail: Send an email] para você
   |-- tipo = followup                        --> [4 Gmail: Send an email] para o lead
```

1. No Make, crie um cenário, clique no **+**, busque **Webhooks** e escolha **Custom webhook**. Clique em **Add**, dê o nome `crm-[negocio]`, salve e copie a URL.
2. No Netlify, crie a variável `MAKE_WEBHOOK_URL` com essa URL (caminho do Passo 3) e publique de novo. Quem tem a URL consegue disparar o seu cenário: ela fica só ali.
3. Com o Make escutando, envie um teste pelo formulário. O módulo reconhece a estrutura: `tipo`, `para`, `assunto`, `html`, `lead`.
4. Adicione o **Router**, com um filtro pelo campo `tipo` em cada rota.
5. Rota de avisos: **Gmail** → **Send an email** para você, com assunto e corpo mapeados do webhook. Os textos saem do **Prompt 5.8**.
6. Rota de follow-up: **Gmail** → **Send an email** para o lead, com você em cópia oculta, como no follow-up da Operator Studio.
7. **Erros:** nas configurações do cenário, guarde as execuções incompletas; no seu perfil do Make, ligue os avisos de erro por e-mail. Se o Make falhar, o lead já está no CRM e aparece na tela Hoje.
8. Ligue o cenário. Se o Make pedir configuração extra para conectar um Gmail pessoal, siga a ajuda do Make sobre conexão com o Google; fazemos juntos na sessão.

WhatsApp (opcional): aviso automático no seu WhatsApp exige a API oficial (WhatsApp Business Cloud API), que montamos na Semana 7. Até lá, o e-mail no celular resolve.

8. Ative o follow-up automático

Escolha **uma** opção. As duas juntas mandam mensagem dobrada.

Opção A (recomendada) — Função agendada do Netlify

É o recurso que envia os lembretes de agendamento da Operator Studio: uma função que roda sozinha num horário fixo, procura quem precisa de mensagem e chama o Make.

1. Cole o **Prompt 5.9**. A regra: etapa Novo lead, Em conversa ou Proposta enviada; consentimento marcado; **Não contatar** desmarcado; último contato há 2 dias ou mais (ou os seus intervalos); menos de 3 follow-ups.
2. Para cada lead, a função escolhe a mensagem (1, 2 ou 3), chama o Make com `tipo = followup` e atualiza o contador e o último contato. Depois da terceira, move para Perdido com motivo "sem resposta".
3. O horário fica no próprio código, em UTC. 9h30 de Brasília é `30 12 * * *`.

4. Depois do deploy, a função aparece na página **Functions** do Netlify como agendada, com a próxima execução. Para testar sem esperar, abra a função e clique em **Run now**.
5. A função agendada tem até 30 segundos por execução e só roda na versão publicada do site. O Claude escreve o código dentro desses limites.

Opção B — Cenário agendado no Make: um cenário diário chama uma rota protegida da sua API (senha guardada na conexão do Make) e envia pelo Gmail. Funciona, mas divide a regra entre Make e site.

Follow-up com um botão. No CRM da Operator Studio, o follow-up depois da análise sai por um botão: o Luccas escolhe os leads do dia, vê a prévia do e-mail e envia; o e-mail sai pelo Make com cópia para ele. Peça um botão assim para mensagens que dependem de você (proposta, pós-atendimento).

Regra de ouro: no máximo 3 mensagens, espaçadas, sempre com saída. Todas terminam com "Se preferir não receber mais mensagens, é só responder SAIR." Quem responder SAIR: abra o lead e marque **Não contatar** na hora.

9. (Opcional) Relatório da reunião com a API do Claude

No CRM da Operator Studio, a transcrição da conversa é colada na ficha do lead e a API do Claude gera o relatório: interno (dores, objeções, interesse) ou para o cliente, que pode virar PDF e e-mail.

1. No Console da Anthropic (platform.claude.com), crie uma chave de API e defina um limite de gasto mensal. Preços na página oficial.
2. No Netlify, crie a variável `ANTHROPIC_API_KEY` (caminho do Passo 3) e publique de novo. A chave nunca vai para o chat, documentos ou código.
3. Cole o **Prompt 5.10**. O Claude cria o campo "Colar transcrição" e uma função em segundo plano (background function), que tem mais tempo para trabalhar.
4. A transcrição vem do Google Meet, se o seu plano do Google oferecer, ou das suas anotações. Avise o cliente que a conversa será transcrita e releia todo relatório antes de enviar.

10. Teste o fluxo inteiro como cliente

Faça do celular, com o seu e-mail e o seu WhatsApp:

1. **Caminho feliz:** formulário enviado, lead em **Hoje** e na etapa Novo lead, consentimento com data e texto, aviso no seu e-mail em menos de 1 minuto.
2. **Duplicado:** mesmo e-mail de novo. Atualiza o lead, não cria outro, e o aviso diz "lead voltou".
3. **Sem consentimento:** o formulário não deixa enviar.
4. **Dados bagunçados:** acento, telefone com parênteses, e-mail com maiúscula. Tudo chega padronizado.
5. **Senha:** errada é recusada, e a página tem a instrução de não indexar.
6. **Make desligado:** desligue o cenário e envie um lead. O lead precisa estar no CRM mesmo sem aviso. Religue o cenário.
7. **Follow-up:** peça ao Claude um lead de teste com último contato de 3 dias atrás, rode **Run now** e confira: e-mail recebido, contador em 1.

8. **Não contatar:** marque no lead de teste e rode de novo. Nada pode sair.
9. **Importação:** 10 clientes importados conferidos contra a lista original.

Registre no Template. Só está pronto quando todos passarem. Depois, apague os leads de teste e crie a rotina: tela **Hoje** todo dia de manhã.

Biblioteca de prompts

Prompt 5.1 — Desenhar o funil de vendas

Quando usar: no início da sessão, antes de construir qualquer coisa.

PROMPT

```
Contexto: Sou dono(a) de [TIPO DE NEGÓCIO] em [CIDADE/ONLINE]. Vendo [PRINCIPAIS
SERVIÇOS/PRODUTOS], ticket médio de [VALOR OU FAIXA]. Os clientes chegam por [CANAIS]. Use
também a base de conhecimento deste Projeto.

Objetivo: Desenhar as etapas do meu funil de vendas, do primeiro contato até o cliente
recorrente, para virar o funil do meu CRM.

Entradas: Meus últimos 10 contatos e o que aconteceu com cada um:
[COLE: INICIAIS, CANAL, O QUE PEDIU, O QUE ACONTECEU, QUANTO TEMPO LEVOU]

Restrições:
- Entre 5 e 7 etapas, com nomes de até 3 palavras.
- Cada etapa com critério de entrada objetivo e observável (um fato, não uma intenção).
- Inclua uma etapa final de perda.
- Não invente taxas de conversão nem médias de mercado.

Formato da resposta:
1. Tabela: Etapa | Código sem acento (ex.: novo, proposta) | Critério de entrada | Próxima
ação padrão | Prazo máximo.
2. De 4 a 6 motivos de perda adequados ao meu negócio.
3. Meus 10 contatos classificados nas etapas, em tabela.
4. Os 3 pontos onde pareço perder mais vendas, separando fato de hipótese.

Validação: Confira se cada contato coube em exatamente uma etapa. Se algum não couber, ajuste
as etapas e explique o ajuste em uma linha.
```

Como conferir: cada contato tem uma etapa só; os critérios são fatos; os prazos cabem na sua rotina; cada etapa tem um código sem acento, que o sistema vai usar.

Prompt 5.2 — Modelo de dados do CRM

Quando usar: depois de aprovar o funil.

PROMPT

```
Contexto: Vou construir um CRM próprio numa página privada do meu site (mesmo repositório do
GitHub, publicado no Netlify). Os dados ficam no Netlify Blobs: uma store "leads" com um JSON
por lead e uma store "crm-arquivos" para anexos. Funil aprovado:
[COLE A TABELA DO PROMPT 5.1]
Perfil na mentoria: [CONSTRUIR / OTIMIZAR / AUTOMATIZAR].

Objetivo: Definir a ficha do lead (campos, tipos, valores permitidos) que todas as funções e a
página vão usar.
```

```
Entradas:
- Campos que o formulário/agendamento envia hoje: [EX.: NOME, WHATSAPP, E-MAIL, SERVIÇO,
MENSAGEM, CONSENTIMENTO]
- O que eu anoto hoje sobre clientes: [LISTE]
- Serviços que vendo: [LISTE]

Restrições:
- Nomes de campo em camelCase, sem acento. Obrigatórios: id, nome, origem, etapa,
consentimento (aceito, data, texto, pagina), ultimoContato, followups, naoContatar,
dataProximaAcao, motivoPerda, criadoEm, atualizadoEm.
- Etapa e origem só aceitam códigos definidos; liste-os.
- LGPD: nada de CPF, nascimento ou endereço, salvo se eu disser que preciso (explique por
quê).
- Diga como evitar duplicados (e-mail ou WhatsApp padronizado).

Formato da resposta:
1. Tabela: Campo | Tipo | Valores permitidos ou formato | Quem preenche (formulário, função,
eu) | Para que serve.
2. Um exemplo de ficha completa em JSON, com dados fictícios.
3. Regras de padronização (telefone +55DDDnúmero, e-mail minúsculo, nome com iniciais
maiúsculas).

Validação: Confira se cada campo é usado numa tela, num filtro, num aviso ou numa decisão.
Remova os que não são e diga quais removeu.
```

Como conferir: os códigos de etapa batem com o funil; nenhum dado sensível sem motivo; o JSON de exemplo usa só dados fictícios.

Prompt 5.3 — API do CRM protegida por senha

Quando usar: com a variável CRM_KEY já criada no Netlify.

PROMPT

```
Contexto: Meu site está no repositório [NOME DO REPOSITÓRIO] no GitHub, publicado no Netlify,
com funções em netlify/functions. A ficha do lead está na base deste Projeto. A senha do CRM
está na variável de ambiente CRM_KEY do Netlify (não me peça a senha).

Objetivo: Criar a Netlify Function netlify/functions/crm.mjs, no caminho /api/crm, que lê e
grava os leads no Netlify Blobs.

Entradas:
- Etapas válidas: [CÓDIGOS DO PROMPT 5.1]
- Ações necessárias: listar leads; criar lead manual; atualizar etapa, notas, valor, próxima
ação, data da próxima ação, motivo de perda e naoContatar; excluir lead manual.

Restrições:
- @netlify/blobs com getStore e consistência forte; acrescente o pacote ao package.json.
- Toda requisição precisa do cabeçalho x-crm-key igual a process.env.CRM_KEY; sem ele ou
diferente, 401 sem nenhum dado.
- Nenhuma senha, chave ou URL de webhook no código.
- Valide e limite cada campo; rejeite etapa fora da lista; atualize atualizadoEm em toda
gravação; respostas com cache-control: no-store.
- Código simples, comentado em português.
```

```
Formato da resposta:
1. O que você vai criar ou alterar, arquivo por arquivo.
2. O código, aplicado no repositório.
3. Como testar depois do deploy: sem senha (401), senha errada (401), senha certa (lista vazia
ou com leads).

Validação: Antes de terminar, releia o código procurando qualquer caminho que devolva dados
sem conferir a senha. Diga o que conferiu.
```

Como conferir: os três testes de senha passaram; não há senha nem URL de webhook no GitHub; o deploy no Netlify terminou sem erro.

Prompt 5.4 — Formulário e agendamento entrando no CRM

Quando usar: para ligar a função da Semana 3 ao CRM.

PROMPT

```
Contexto: A função [NOME DO ARQUIVO, EX.: netlify/functions/contato.mjs] recebe o formulário
[E/OU O AGENDAMENTO] do meu site. Agora ela precisa alimentar o CRM. Ficha do lead e códigos
de origem e etapa estão na base deste Projeto. O webhook do Make está na variável
MAKE_WEBHOOK_URL.

Objetivo: Atualizar a função para gravar cada contato como lead no Netlify Blobs (store
"leads") e avisar pelo Make.

Entradas:
- Campos do formulário: [LISTE]
- Texto exato do checkbox de consentimento: [COLE O TEXTO DO PROMPT 5.5]
- Meu e-mail para avisos: [E-MAIL]
- Link do CRM: [SEUDOMINIO]/crm/

Restrições:
- Ordem: validar → gravar no Blobs → responder ao cliente → chamar o Make. Se o Make falhar,
o lead fica gravado e o erro vai para o log.
- Sem consentimento, sem gravação (erro claro). Guarde aceito, data e hora (ISO), texto exato
e página.
- Campo escondido preenchido (robô): responda ok e não grave.
- Padronize telefone (+55DDDnúmero), e-mail (minúsculo) e nome.
- Duplicado (mesmo e-mail ou WhatsApp): atualize o lead, acrescente a mensagem nas notas,
atualize ultimoContato, aviso "lead_voltou"; se estava em Perdido, volte para Novo lead. Lead
novo: etapa novo, aviso "novo_lead".
- Payload para o Make: tipo, para, assunto, html (curto, legível no celular), lead (só o
necessário).
- Mantenha o resto do comportamento atual.

Formato da resposta:
1. Resumo das mudanças.
2. Código aplicado no repositório.
3. Roteiro de teste: lead novo, duplicado, sem consentimento, robô.

Validação: Confira que nenhum caminho grava lead sem consentimento e que nenhum segredo está
no código.
```

Como conferir: o teste aparece na gaveta `leads` do Blobs; o duplicado não cria ficha nova; sem consentimento nada é gravado.

Prompt 5.5 — Consentimento e Política de Privacidade (LGPD)

Quando usar: para o checkbox do formulário e a página /privacidade.

PROMPT

```
Contexto: Meu negócio [NOME DO NEGÓCIO], [TIPO DE NEGÓCIO], CNPJ/CPF [NÚMERO OU "INFORMAR
DEPOIS"], coleta dados pelo formulário do site para responder pedidos e fazer follow-up. Os
dados ficam no meu site, hospedado no Netlify (Netlify Blobs), os e-mails passam pelo Make.com
e saem pelo Gmail. [SE FOR USAR: transcrições de reunião são processadas pela API do Claude,
da Anthropic.] Contato por [E-MAIL / WHATSAPP].

Objetivo: Redigir (1) o texto do checkbox de consentimento e (2) uma Política de Privacidade
simples, alinhada à LGPD (Lei 13.709/2018), para a página /privacidade.

Entradas:
- Dados coletados: [LISTE]
- Finalidade: [EX.: RESPONDER O PEDIDO, ENVIAR ORÇAMENTO, ATÉ 3 MENSAGENS DE ACOMPANHAMENTO]
- Prazo de guarda: [EX.: 24 MESES APÓS O ÚLTIMO CONTATO]
- E-mail para pedidos do titular: [E-MAIL]

Restrições:
- Checkbox com até 35 palavras: quem entra em contato, por quais canais, para quê, com link
para a política.
- Política com até 600 palavras: quem somos, dados, finalidade, base legal, com quem
compartilhamos (ferramentas que processam os dados), prazo de guarda, direitos do titular,
como parar de receber mensagens, contato.
- Não afirme certificações, servidores no Brasil ou medidas de segurança que eu não informei.
- No fim, uma nota só para mim com os pontos que pedem revisão de um advogado.

Formato da resposta: 1) Texto do checkbox. 2) Política pronta para a página do site. 3) Nota
de revisão.

Validação: Confira se tudo o que a política afirma bate com as entradas e se não há promessa
que eu não consiga cumprir.
```

Como conferir: o texto bate com o que você faz com os dados; há um e-mail para pedidos; a nota de revisão foi lida. É ponto de partida, não parecer jurídico.

Prompt 5.6 — Página do CRM e anexos

Quando usar: com a API do Prompt 5.3 funcionando.

PROMPT

```
Contexto: A API /api/crm do meu site já funciona com o cabeçalho x-crm-key. Ficha do lead,
etapas e Brand Kit estão na base deste Projeto. Eu uso o CRM principalmente no celular.

Objetivo: Criar a página privada crm/index.html e a função netlify/functions/crm-files.mjs
para anexos.
```

```
Entradas:
- Etapas e nomes de exibição: [COLE]
- Mensagens rápidas de WhatsApp que eu mando sempre: [EX.: PRIMEIRO CONTATO, ENVIO DE
PROPOSTA, CONFIRMAÇÃO]
- Meu nome e o nome do negócio: [NOMES]

Restrições:
- Login com senha enviada no cabeçalho x-crm-key, nunca na URL; guardada só neste aparelho,
com botão Sair.
- Telas: Hoje (próxima ação hoje ou atrasada), Funil (colunas no computador, abas no celular,
com contagem), Busca, Novo lead manual.
- Ficha: contato, origem, consentimento, botões de etapa, notas, valor, próxima ação e data,
motivo de perda, Não contatar, WhatsApp (wa.me com mensagem pronta) e e-mail.
- Anexos: crm-files grava na store "crm-arquivos" (PDF ou imagem, até 5 MB), com a mesma
senha; abrir e remover na ficha.
- noindex na página e X-Robots-Tag para /crm/* no netlify.toml.
- Brand Kit, fundo claro, botões grandes para o dedo, erros em português claro.

Formato da resposta:
1. Lista de arquivos criados ou alterados.
2. Código aplicado no repositório.
3. Roteiro de teste no celular, item por item.

Validação: Confira que nenhuma tela mostra dados antes do login e que todo pedido à API leva a
senha no cabeçalho.
```

Como conferir: senha errada não mostra nada; o funil mostra os leads nas etapas certas; um PDF anexado abre depois de sair e entrar de novo; a página tem noindex.

Prompt 5.7 — Limpar o CSV e criar a importação

Quando usar: para trazer os clientes atuais para o CRM.

PROMPT

```
Contexto: Vou importar meus clientes e contatos atuais para o CRM do meu site. Anexei o
arquivo CSV exportado de [ORIGEM]. A ficha do lead está na base deste Projeto.

Objetivo: (1) Limpar e padronizar a lista e me devolver um CSV pronto. (2) Criar no CRM um
botão "Importar CSV" com prévia antes de gravar.

Entradas:
- Colunas finais com os nomes exatos da ficha: nome, whatsapp, email, origem, servico, etapa,
ultimoContato, consentimento, notas
- Critério de consentimento: marcar SOMENTE para [EX.: CLIENTES QUE AUTORIZARAM CONTATO POR
ESCRITO]; anotar em notas como autorizou.
- Etapa padrão dos antigos: cliente. Origem: importado.

Restrições:
- Telefone +55DDDnúmero (sem DDD: mantenha e anote "conferir DDD"). E-mail minúsculo (inválido
fica vazio, com nota). Nome com iniciais maiúsculas. Datas AAAA-MM-DD.
- Remova duplicados (telefone ou e-mail); em conflito, guarde os dois valores em notas. Não
invente nenhum dado.
- Botão de importação: lê o CSV no navegador, mostra prévia (novos, já existentes, com
```

```
problema), só grava depois da minha confirmação, envia em lotes pela /api/crm com a senha e
nunca duplica quem já está no CRM.

Formato da resposta:
1. CSV limpo para download (UTF-8, vírgula, cabeçalho na primeira linha).
2. Resumo: linhas recebidas, linhas no CSV, duplicados removidos, linhas com "conferir".
3. Tabela só com as linhas que precisam de revisão humana e o motivo.
4. Código do botão de importação e da ação nova na API, aplicado no repositório.

Validação: Confira que toda linha tem nome e pelo menos um contato e que nenhum valor de etapa
ou origem está fora da lista.
```

Como conferir: 10 linhas aleatórias batem com a lista original; o total fecha; todo "conferir" foi resolvido antes de importar; importar o mesmo arquivo duas vezes não duplica ninguém.

Prompt 5.8 — Avisos para o dono

Quando usar: para os e-mails que a função manda pelo Make.

PROMPT

```
Contexto: Sou [SEU NOME], de [NOME DO NEGÓCIO]. Meu site me avisa de cada lead: a função monta
o e-mail em HTML e o Make envia pelo Gmail. Use o Brand Kit deste Projeto.

Objetivo: Escrever três avisos: (1) novo lead, (2) lead que voltou, (3) erro (o lead chegou,
mas algo falhou).

Entradas: Campos disponíveis: {{nome}}, {{whatsapp}}, {{email}}, {{servico}}, {{mensagem}},
{{origem}}, {{link_whatsapp}}, {{link_crm}}, {{data_hora}}, {{erro}}. Meu prazo de resposta:
[EX.: 2 HORAS ÚTEIS].

Restrições:
- Assunto com até 60 caracteres, começando por "Novo lead", "Lead voltou" ou "Erro no CRM",
seguido do nome.
- Corpo para o celular: até 8 linhas, um campo por linha, botões para WhatsApp e CRM, uma
linha de ação ("Responder até [HORÁRIO]"). HTML simples com estilos inline, cores da marca,
fundo claro.
- O aviso de erro traz todos os dados do lead, para eu atender mesmo sem abrir o CRM.
- Mantenha os campos entre chaves duplas exatamente como listei.

Formato da resposta: Para cada aviso: Assunto, versão em texto, HTML.

Validação: Confira se cada aviso se entende em 5 segundos e se nenhum campo foi renomeado.
```

Como conferir: os campos batem com os da função; os botões abrem o WhatsApp e o CRM no celular.

Prompt 5.9 — Follow-up automático com função agendada

Quando usar: para criar a sequência de follow-up e a função que envia.

PROMPT

```
Contexto: Sou [SEU NOME], de [NOME DO NEGÓCIO]. Meu CRM guarda os leads no Netlify Blobs
(store "leads"), com etapa, consentimento, naoContatar, ultimoContato e followups. Os e-mails
saem pelo webhook do Make (variável MAKE_WEBHOOK_URL) com tipo "followup". Siga o tom de voz
deste Projeto.

Objetivo: (1) Escrever 3 mensagens de follow-up. (2) Criar a função agendada
netlify/functions/followup.mjs que envia a mensagem certa para cada lead.

Entradas:
- Etapas que recebem follow-up: [EX.: NOVO, CONVERSA, PROPOSTA]
- Intervalos: mensagem 1 após [2] dias, mensagem 2 após [5], mensagem 3 após [10]
- Horário de envio: [EX.: 9H30] no fuso [EX.: AMERICA/SAO_PAULO]
- Próximo passo desejado: [EX.: RESPONDER COM O MELHOR HORÁRIO / AGENDAR EM [LINK]]

Restrições das mensagens:
- 1: retoma o pedido com uma pergunta fácil. 2: informação útil sobre o serviço, sem inventar
resultados ou números. 3: encerra com educação e diz que não vai insistir.
- Até 90 palavras, uma chamada, sem urgência falsa nem desconto não autorizado. Todas terminam
com: "Se preferir não receber mais mensagens, é só responder SAIR." Versão WhatsApp manual com
até 50 palavras.

Restrições da função:
- Cron em UTC no config da função; explique a conversão do meu horário.
- Só envia se: etapa na lista, consentimento aceito, naoContatar falso, e-mail presente,
followups < 3, intervalo vencido.
- Depois: followups + 1, ultimoContato = agora; na terceira, etapa perdido e motivoPerda "sem
resposta".
- Marque o envio antes de chamar o Make, para nunca mandar duas vezes.
- Caiba em 30 segundos (lotes, se precisar) e registre no log um resumo de cada execução.

Formato da resposta:
1. Tabela: Mensagem | Quando | Assunto | E-mail | WhatsApp.
2. Código aplicado no repositório.
3. Como testar com "Run now" no painel do Netlify.

Validação: Leia as três mensagens como se fosse o cliente e reescreva a que soar como
cobrança. Confira no código que ninguém com naoContatar ou sem consentimento recebe nada.
```

Como conferir: o teste com Run now mandou só para o lead de teste; o contador subiu; um lead marcado como Não contatar não recebeu nada.

Prompt 5.10 — Relatório da reunião com a API do Claude (opcional)

Quando usar: perfil Automatizar, com a variável ANTHROPIC_API_KEY criada no Netlify.

PROMPT

```
Contexto: Meu CRM roda no meu site (Netlify Functions e Netlify Blobs). Depois de cada reunião
com um lead ou cliente, quero colar a transcrição na ficha e receber um relatório. A chave da
API do Claude está na variável ANTHROPIC_API_KEY do Netlify (não me peça a chave).

Objetivo: Criar o campo "Colar transcrição" na ficha do lead e uma função em segundo plano
(background function) que chama a API do Claude e grava o relatório na ficha.
```

```
Entradas:
- Tipos de relatório: (a) interno: necessidade, dores, objeções, nível de interesse, próximo passo sugerido; (b) para o cliente: resumo, decisões, próximos passos, em tom [TOM DA MARCA].
- Meu negócio e serviços: [DESCRIÇÃO CURTA]

Restrições:
- Transcrição numa store própria ("transcricoes"), ligada ao id do lead; relatório na ficha com status (gerando, pronto, erro), atualizando sozinho na página.
- A função confere a senha do CRM antes de tudo.
- API de Mensagens da Anthropic, com o modelo recomendado na documentação oficial e resposta em campos fixos.
- O relatório não inventa nada fora da transcrição e marca trechos incertos.
- Em erro, mostrar a mensagem na ficha e permitir tentar de novo.

Formato da resposta:
1. Arquivos criados ou alterados.
2. Código aplicado no repositório.
3. Como testar com uma transcrição fictícia de 1 página.

Validação: Confira que a chave só é lida de process.env e que nenhum relatório é enviado ao cliente sem eu clicar para enviar.
```

Como conferir: o relatório da transcrição fictícia não traz nada que não estava nela; a chave não aparece no repositório; erro aparece na ficha com opção de tentar de novo.

Exercício prático

Construir: funil com 5 etapas, CRM em /crm/ com senha e telas Hoje e Funil, formulário entrando no CRM com consentimento, clientes atuais importados, aviso por e-mail pelo Make e follow-up pela função agendada, com 2 mensagens.

Otimizar: tudo do Construir, mais duplicado com aviso "lead voltou", anexos, mensagens rápidas de WhatsApp, motivos de perda e follow-up com 3 mensagens. Se a lista de clientes estava noutra ferramenta, importe pelo CSV e pare de usar a antiga.

Automatizar: tudo do Otimizar, mais agendamento do site entrando no CRM com a data, botão de follow-up com prévia (como o da Operator Studio), relatório de reunião pela API do Claude e um resumo semanal por e-mail (função agendada às segundas) com novos leads, propostas abertas e motivos de perda.

Critério de conclusão: pelo celular, o formulário enviado aparece na tela Hoje em menos de 1 minuto, com consentimento registrado, e o aviso chega. Os testes de duplicado, senha, Make desligado, follow-up e Não contatar passaram. Os clientes atuais estão no CRM.

Checklist de qualidade

- [ ] Funil de 5 a 7 etapas, com critério objetivo e o mesmo código em todas as peças do sistema.
- [ ] `CRM_KEY`, `MAKE_WEBHOOK_URL` e (se usar) `ANTHROPIC_API_KEY` estão só nas variáveis de ambiente do Netlify; nenhum valor aparece no GitHub, no chat ou em documentos.
- [ ] A API responde 401 sem senha ou com senha errada.
- [ ] A página /crm/ tem noindex e não aparece no Google.
- [ ] O checkbox de consentimento é obrigatório, vem desmarcado e tem link para a Política de Privacidade.
- [ ] Cada lead do site tem consentimento com data, texto exato e página registrados.
- [ ] Lead é gravado antes do aviso: com o Make desligado, o lead continua chegando ao CRM.
- [ ] Duplicado atualiza o lead existente, não cria outro.
- [ ] Os clientes atuais foram importados e 10 linhas foram conferidas com a lista original.
- [ ] Só recebe follow-up automático quem tem consentimento e não está marcado como Não contatar.
- [ ] Todo follow-up tem a saída "responda SAIR", e quem pediu foi marcado na hora.
- [ ] Relatório gerado pela API do Claude só sai depois da sua revisão.
- [ ] A função agendada aparece no Netlify com a próxima execução no horário certo.
- [ ] O CSV e as conversas com a lista de clientes foram apagados; os leads de teste também.

Template do entregável

Ficha do CRM e da captação automática — [Nome do negócio]

Data: __ · **Perfil:** _ · ***Endereço do CRM:*** ___/crm/

1. Funil

ETAPA	CÓDIGO	CRITÉRIO DE ENTRADA	PRÓXIMA AÇÃO	PRAZO
_____	_____	_____	_____	_____
_____	_____	_____	_____	_____
_____	_____	_____	_____	_____
_____	_____	_____	_____	_____
_____	_____	_____	_____	_____

Motivos de perda: _____

2. Onde as coisas estão

PEÇA	ARQUIVO OU LUGAR
Repositório no GitHub	_____
Página do CRM (noindex conferido)	crm/index.html
API do CRM	netlify/functions/crm.mjs → /api/crm
Anexos	netlify/functions/crm-files.mjs
Entrada de leads	netlify/functions/_____
Follow-up (horário UTC e local)	netlify/functions/followup.mjs · _____
Stores no Netlify Blobs	leads · crm-arquivos · _____

3. Variáveis de ambiente (só os nomes, nunca os valores): CRM_KEY · MAKE_WEBHOOK_URL · ANTHROPIC_API_KEY (opcional). Criadas em: _____

4. Make

CENÁRIO	GATILHO	ROTAS	ENVIA POR	EXECUÇÕES INCOMPLETAS ATIVAS?
_____	Custom webhook	novo_lead · lead_voltou · erro · followup	Gmail	_____

5. Follow-up

MENSAGEM	QUANDO	ASSUNTO
1	_____	_____
2	_____	_____
3	_____	_____

6. LGPD

- Texto do checkbox: _____
- Link da Política de Privacidade: _____
- Prazo de guarda: **__ · E-mail para pedidos do titular: ___**

7. Testes

CASO	RESULTADO ESPERADO	PASSOU?	DATA
Caminho feliz no celular	Lead em Hoje + aviso em menos de 1 min	_____	_____
Duplicado	Atualiza e avisa "lead voltou"	_____	_____
Sem consentimento	Formulário não envia	_____	_____
Senha errada	Nada aparece	_____	_____
Make desligado	Lead gravado mesmo sem aviso	_____	_____
Follow-up (Run now)	E-mail recebido, contador em 1	_____	_____
Não contatar	Nada enviado	_____	_____
Importação	10 linhas conferidas	_____	_____

8. Rotina

- Horário diário da tela Hoje: _____
- Quem cobre na minha ausência (e como recebe a senha com segurança): _____
- Revisão semanal do funil: _____

Erros comuns

- **Construir antes de fechar o funil:** sobram campos que ninguém usa e etapas que ninguém entende. Feche o funil com o Prompt 5.1 primeiro.

- **Senha ou URL do webhook no código:** tudo o que vai para o GitHub fica no histórico. Segredo só nas variáveis de ambiente do Netlify. Se vazou, troque a senha no Netlify e publique de novo.
- **Mudar uma variável e não publicar:** a mudança só vale depois de um novo deploy.
- **Avisar antes de gravar:** se o Make falha, o lead some. A função grava primeiro, avisa depois.
- **Follow-up para quem já respondeu:** acontece quando a conversa anda no WhatsApp e o CRM fica parado. Respondeu, mude a etapa e a data do último contato.
- **Importar todo mundo com consentimento marcado:** quem nunca autorizou não recebe mensagem automática.

Para a próxima semana

Na Semana 6 você redesenha um processo crítico e constrói um sistema interno para ele. Traga os 3 processos que mais tomam o seu tempo (atendimento do pedido, onboarding, entrega do serviço, compras, escala da equipe), um exemplo real recente de cada um e os nomes de quem participa. Use a tela Hoje a semana toda e anote o que travou: o CRM vai se ligar ao sistema interno.